COMP4442 Service and Cloud Computing
Lab 4

Weiyang MA

Mar.25, 2026

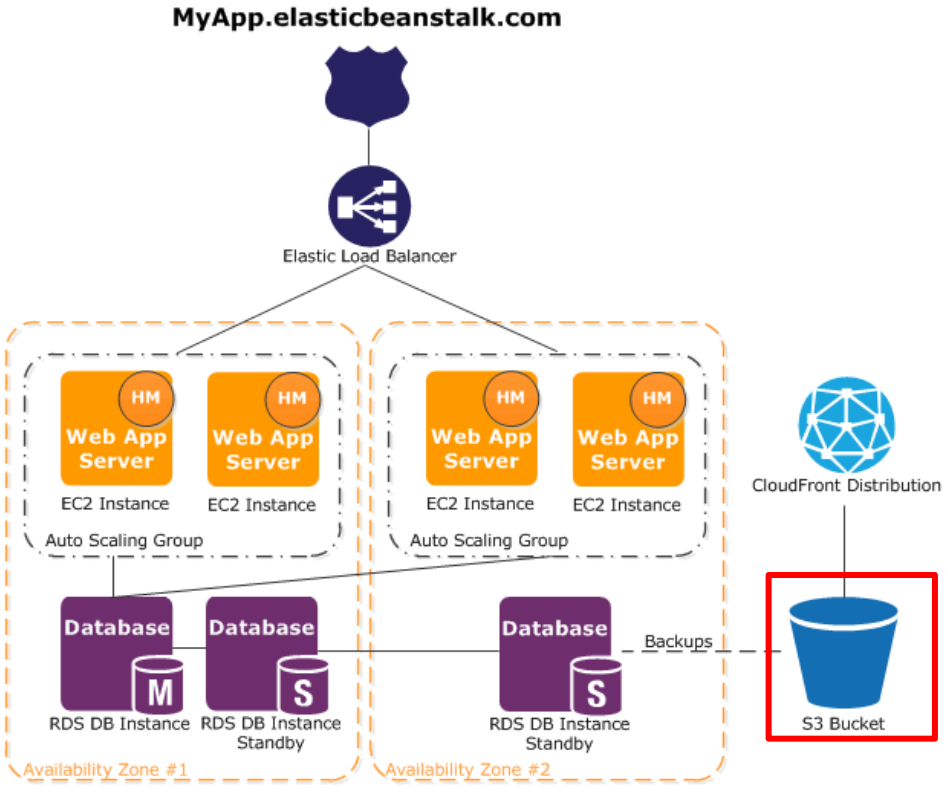
Outline

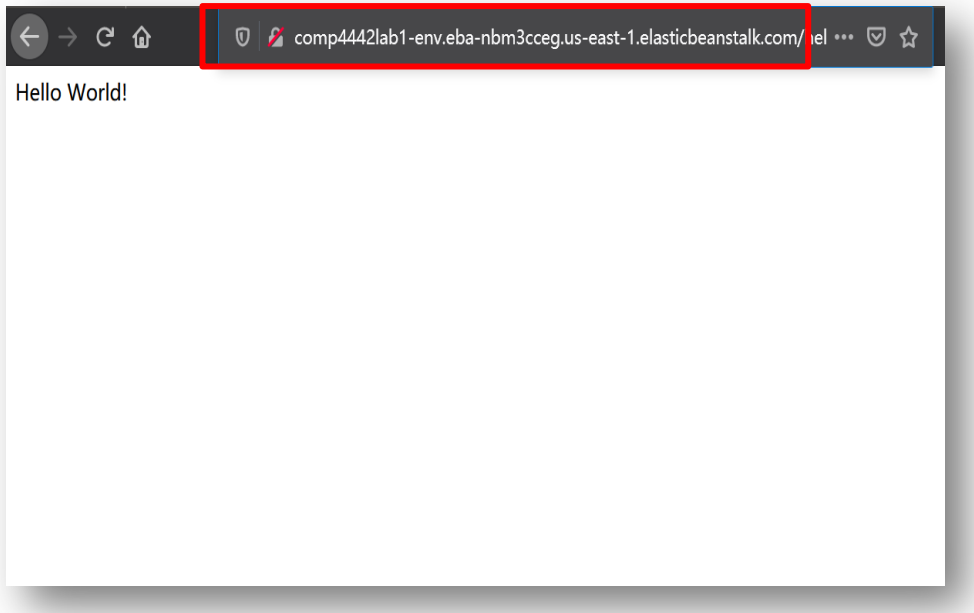
- Review of Labs

- How to develop dynamic web app with AWS Beanstalk
  - Structure of flask project
  - Create static webpage with flask
  - Create dynamic web app with flask
  - Deploy code in AWS Beanstalk

Review of Labs

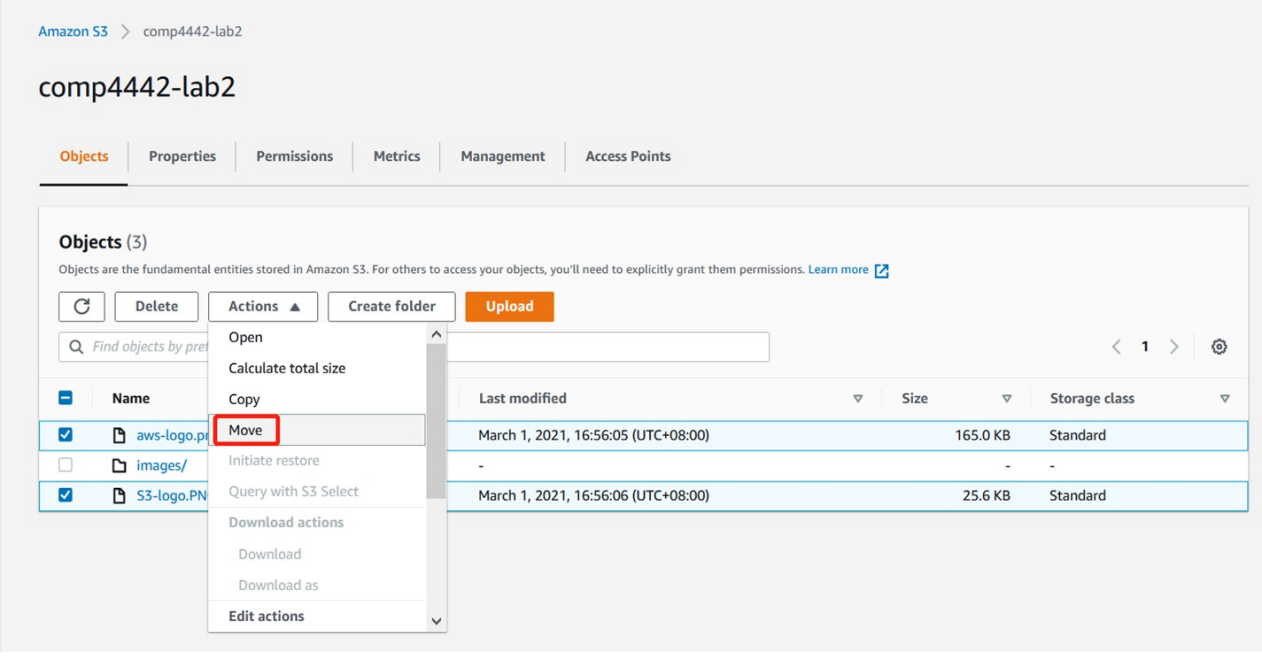
- We learned how to use AWS services to develop a simple web application.
  - Use Elastic Beanstalk with simple configuration

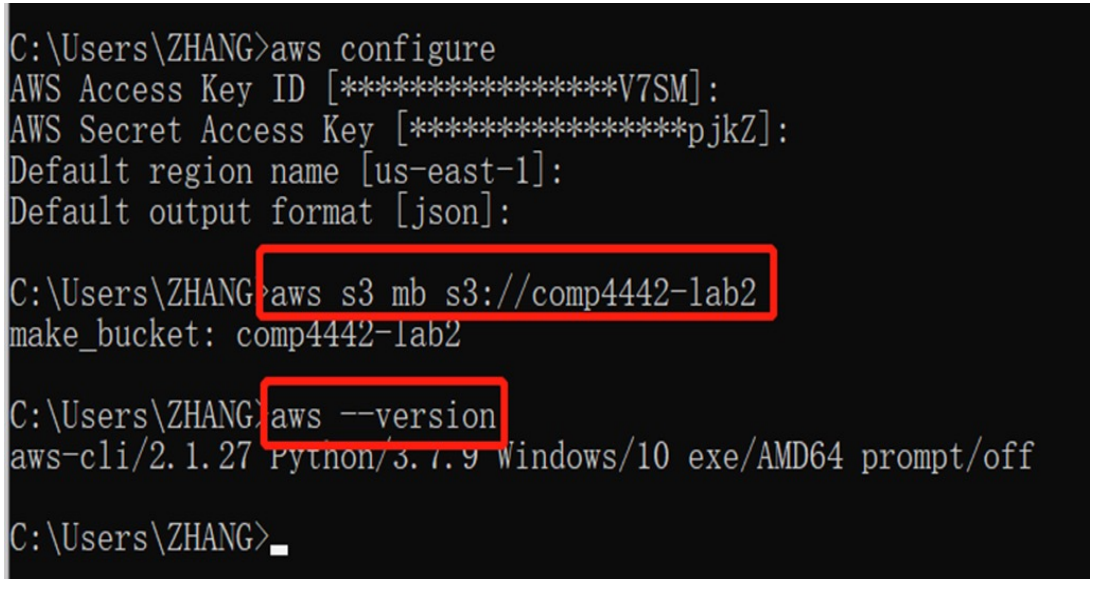




Review of Labs

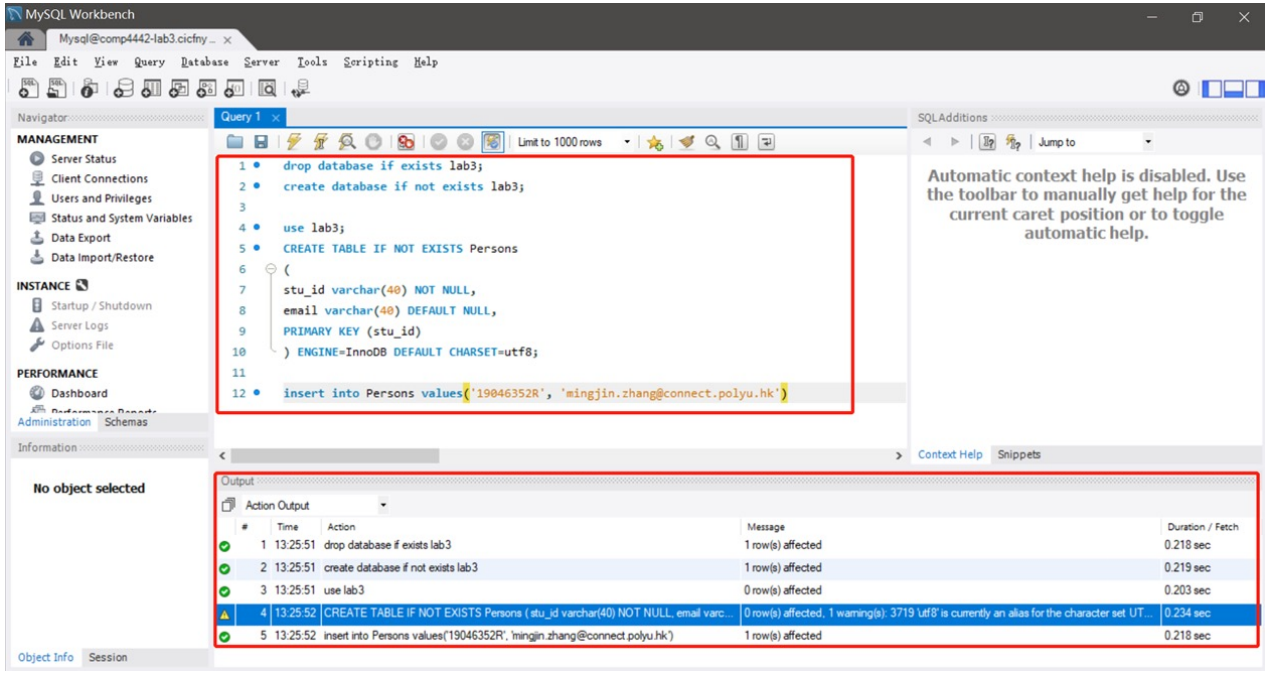
- We learned how to use AWS S3 to manage files (upload, delete, move, etc.)
  - Via AWS console
  - Via AWS CLI





Review of Labs

- We learned how to use AWS RDS to manage structural data
  - Via MySQL Workbench
  - Via python




```
import mysql.connector

mydb = mysql.connector.connect(
host = '<hostname>',
user = '<master username>',
port = '3306',
database = 'lab3',
passwd = '<master password>'

)

mycursor = mydb.cursor()
mycursor.execute("select * from Persons")
myresult = mycursor.fetchall()

for x in myresult:
    print(x)
```

Python Flask

- Flask is a popular lightweight web application framework. It is designed to make getting started quick and easy.
- Lab Preparation
  - Create an empty directory *lab4* and go to the directory `cd lab4`
  - Install the virtual environment `pip install virtualenv`
  - Create a virtual environment `virtualenv lab4`
  - Activate the virtual environment
    - `lab4\Scripts\activate` (windows)
    - `Source lab4/bin/activate` (macOS)
  - Install flask `pip install flask`
  - Install mysql-connector `pip install mysql-connector-python`

Python Flask

- Structure of a flask project

```
C:\Users\ZHANG\Desktop\flask_project
│  application.py  ➡ Programming logic control
│
├─static  ➡ Folder consists static files, e.g., images, css, js files (optional)
│     style.css
│
└─templates ➡ *html files in templates are for front-end view (optional)
      home.html
      list.html
      result.html
      student.html
```

Create Static Webpage with Flask

- Create the *application.py* file with the following code

```
from flask import Flask

application = Flask(__name__)

@application.route("/")
def index():
    return "Your Flask App Works!"

@application.route("/hello")
def hello():
    return "Hello World!"


if __name__ == "__main__":
    application.run(port=5000, debug=True)
```

From flask package import Flask class

Instantiate the class to create the application object

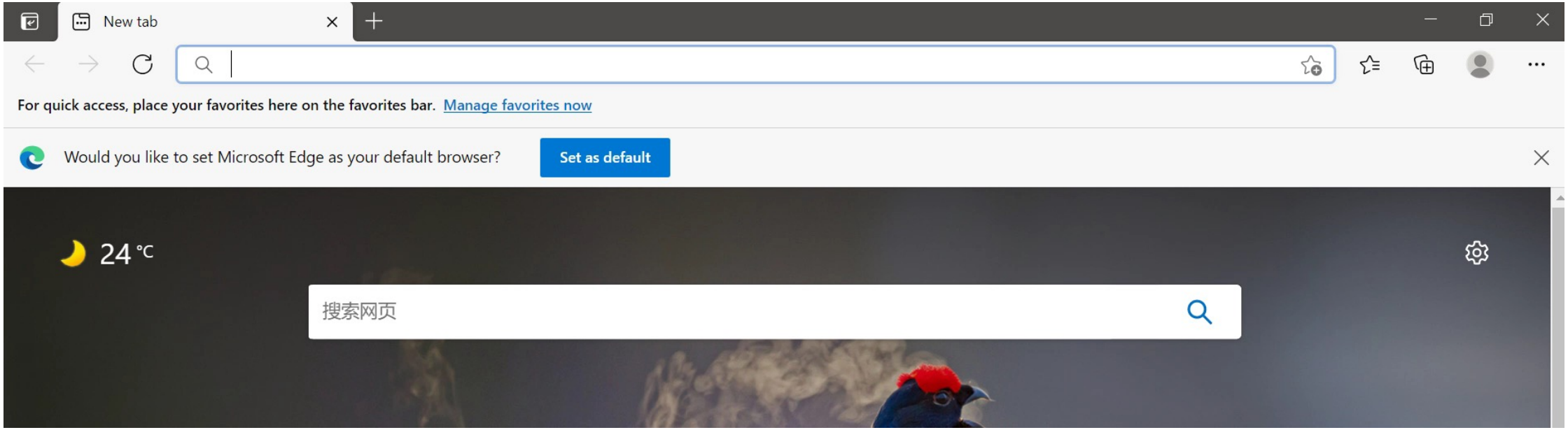
Server will call the index() to response to http://127.0.0.1/5000

Server will call the hello() to response to http://127.0.0.1/hello

Program entry

Create Dynamic Web App with Flask

- We will develop a student information collection web application
  - A HTML form to enable students input information
  - A database to store the input information
  - A HTML table to query and display the input information

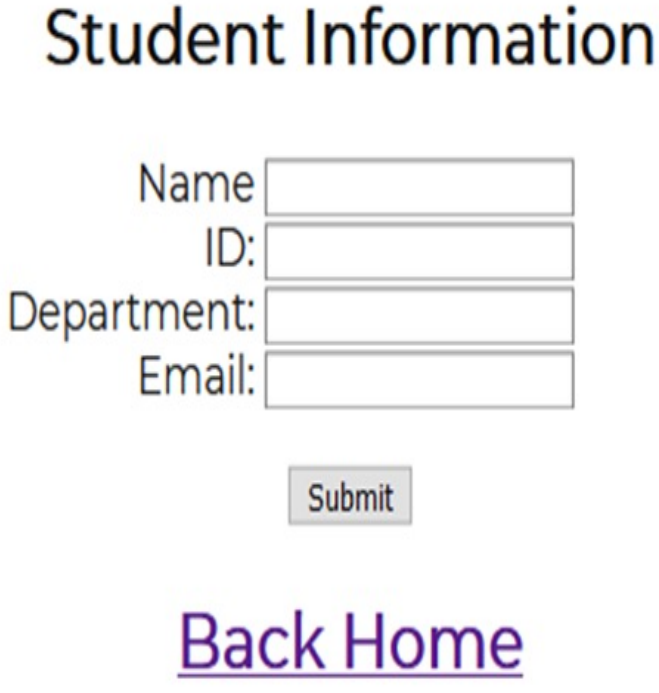


Create Dynamic Web App with Flask

- First, we create the student information form

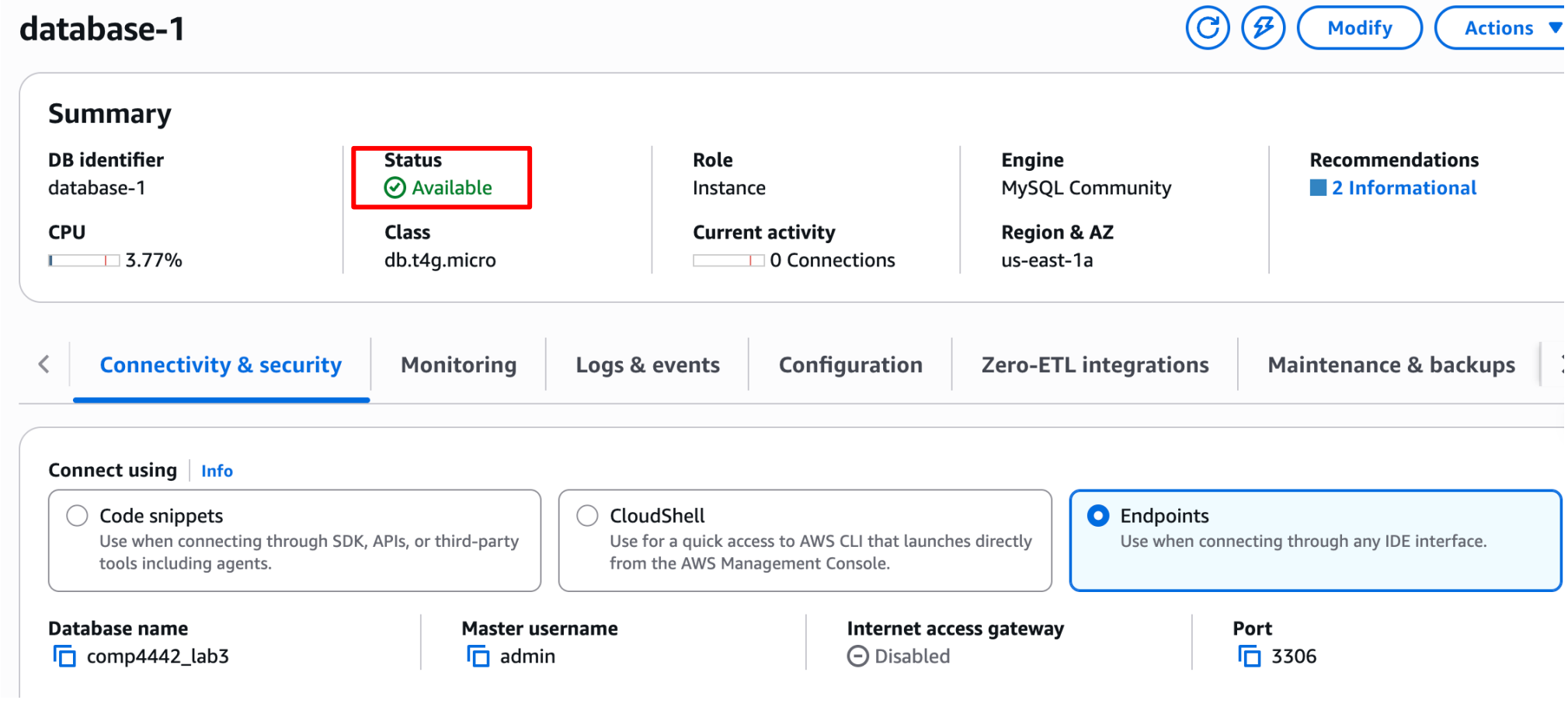
/addrec to handle the post request

```html
<!DOCTYPE html>

<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
<title>Student Information Collection</title>
<link rel="stylesheet" type="text/css" href="../static/style.css">
<link rel="stylesheet" href="https://fonts.googleapis.com/css?family=Montserrat">

</head>
   <body>

      <form action = "{{ url_for('addrec') }}" method = "POST">
         <h2>Student Information</h2>
         <p style="text-align:right; margin:0 35% 0 0">Name <input type = "text" name = "Name" /></p>
         <p style="text-align:right; margin:0 35% 0 0">ID: <input type = "text" name = "ID" /></p>
         <p style="text-align:right; margin:0 35% 0 0">Department: <input type = "text" name = "Department
         " /></p>
         <p style="text-align:right; margin:0 35% 0 0">Email: <input type = "text" name = "Email" /></p>
         <p><input type = "submit" value = "Submit" /></p>
      </form>


   </body>
</html>
```



Prepare the database

- Check the database connection setting



Prepare the database

- Check the database connection setting

Connectivity & security

Endpoint & port

Endpoint
database-1.cfonvzfkm37u.us-ea

Port
3306

Networking

Availability Zone
us-east-1a

VPC
vpc-019e7e254fb06eb57

Subnet group
default-vpc-019e7e254fb06eb57

Subnets
subnet-0414394ce8650718b
subnet-0f7f7a61dff5eda9e
subnet-060a82e3c1a1e3774
subnet-0a225a7354d07ca2e
subnet-0ee1c84fcb79fd30a
subnet-0e9083b80de2fa2ea

Security

VPC security groups
0.0.0.0/0 (sg-01e0682985c8b0990)
Active

Publicly accessible
Yes

Certificate authority Info
rds-ca-rsa2048-g1

Certificate authority date
May 26, 2061, 07:34 (UTC+08:00)

DB instance certificate expiration date
March 06, 2027, 16:49 (UTC+08:00)

Create Dynamic Web App with Flask

- Create a new table in the database we created in lab-03(Refer to Lab-3)

```
drop database if exists lab4;
create database if not exists lab4;

use lab4;
CREATE TABLE IF NOT EXISTS Students
(
Name varchar(40) NOT NULL,
ID varchar(40) NOT NULL,
Department varchar(40) NOT NULL,
Email varchar(40) DEFAULT NULL,
PRIMARY KEY (ID)
) ENGINE=InnoDB DEFAULT CHARSET=utf8;
```

You can run this code in the software introduced in Lab-3 such as MySQL Workbench and Table Plus.

Create Dynamic Web App with Flask

- Create a new table by python(uploaded as init.py in BlackBoard)

```python
import mysql.connector

try:
    db = mysql.connector.connect(
        host="database-1.cfonvzfkm37u.us-east-1.rds.amazonaws.com",
        user="admin",
        port="3306",
        password="xxxxxx"
    )
    cursor = db.cursor()
    sql_commands = [
        "DROP DATABASE IF EXISTS lab4;",
        "CREATE DATABASE IF NOT EXISTS lab4;",
        "USE lab4;",
        """
        CREATE TABLE IF NOT EXISTS Students (
            Name varchar(40) NOT NULL,
            ID varchar(40) NOT NULL,
            Department varchar(40) NOT NULL,
            Email varchar(40) DEFAULT NULL,
            PRIMARY KEY (ID)
        ) ENGINE=InnoDB DEFAULT CHARSET=utf8;
        """
    ]
```

```python
    for command in sql_commands:
        cursor.execute(command)
        print(f"success: {command.split()[0]}...")
    print("lab4 database and Students table established")

    print("--- database ---")
    cursor.execute("SHOW DATABASES")
    for (db_name,) in cursor:
        print(f"{db_name}")


except mysql.connector.Error as err:
    print(f"error: {err}")

finally:
    if 'db' in locals() and db.is_connected():
        cursor.close()
        db.close()
```

Create Dynamic Web App with Flask

- Implement the response function to insert student information to the AWS RDS database

```
@application.route('/addrec',methods = ['POST', 'GET'])
def addrec():
    if request.method == 'POST':
        Name = request.form['Name']
        ID = request.form['ID']
        Department = request.form['Department']
        Email = request.form['Email']

        mydb = db_connection()
        cur = mydb.cursor()
        info = "insert into Students values('{}','{}','{}','{}')".format(Name, ID, Department, Email)
        cur.execute(info)

        mydb.commit()
        msg = "Record successfully added"

        return render_template("result.html",msg = msg)
        mydb.close()

def db_connection():
    mydb = mysql.connector.connect( host = 'comp4442-lab3.cicfnyxayefu.us-east-1.rds.amazonaws.com',
    user = 'admin',
    port = '3306',
    database = 'lab4',
    passwd = '12345678')

    #print("successfully connect to the database")

    return mydb
```

Parser the user input

Add information to database

Create Dynamic Web App with Flask

- Query the information on the RDS database

```html
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
<title>Show Student Information</title>
<link rel="stylesheet" type="text/css" href="/static/style.css">
</head>
   <body>

      <h2>Student Information List</h2>
      <table border=1 style="margin:5% 0 0 15%">
         <thead>
            <tr>
               <th>Name</th>
               <th>ID</th>
               <th>Department</th>
               <th>Email</th>
            </tr>
         </thead>
         <tbody>
            {% for result in results %}
            <tr>
               <td>{{result[0]}}</td>
               <td>{{result[1]}}</td>
               <td>{{result[2]}}</td>
               <td>{{result[3]}}</td>
            </tr>
         {% endfor %}
         </tbody>

      </table>

      <h2><a href = "/">Back Home</a></h2>


   </body>
</html>
```

```python
@application.route('/list')
def list():
    mydb = db_connection()

    cur = mydb.cursor()
    cur.execute("select * from Students")

    myresult = cur.fetchall()
    for result in myresult:
        print(result)

    return render_template("list.html", results = myresult)
```

Create Dynamic Web App with Flask

- Check the student information at "*http://127.0.0.1:5000/list*".

Student Information List

Name	ID	Department	Email
Mingjin	19046352R	CS	mingjin.zhang@connect.polyu.hk
Yinfeng	666666	CS	yinfeng.cao@connect.polyu.hk
Qianyi	777777	CS	qianyi@polyu.hk
zhang	888888	AIoT	zhang@polyu

Back Home

Deploy Code in AWS Beanstalk

- To deploy the code, we need to configure the requirements.txt.
  - `pip freeze > requirement.txt`
- Pack the following files and directories into a ZIP file.

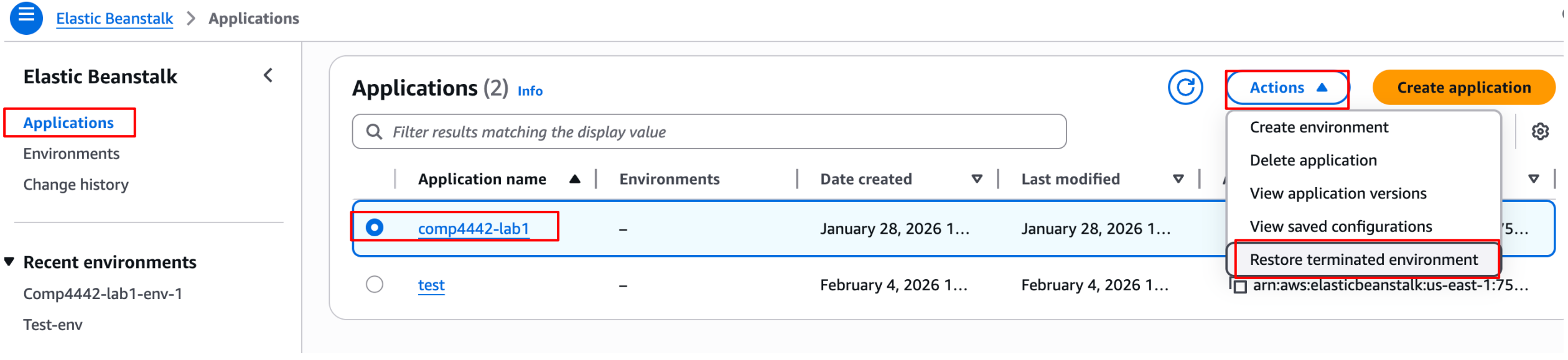
requirements.txt

```
blinker==1.9.0
click==8.3.1
Flask==3.1.3
itsdangerous==2.2.0
Jinja2==3.1.6
MarkupSafe==3.0.3
mysql-connector-python==9.6.0
Werkzeug==3.1.7
```

Name	Date modified	Type	Size
static	4/3/2021 8:44 PM	File folder	
templates	4/3/2021 8:44 PM	File folder	
application.py	4/3/2021 9:16 PM	Python File	2 KB
requirements.txt	4/2/2021 10:05 PM	Text Document	1 KB

Deploy Code in AWS Beanstalk

- Restore Beanstalk environment and upload the ZIP file as we do in lab-01.(Not recommended)



Deploy Code in AWS Beanstalk

- Or you can create a new Beanstalk environment and upload the ZIP file as we do in lab-01.

Elastic Beanstalk is launching your environment. This will take a few minutes.

Comp4442-lab4-env Info

Actions | Upload and deploy

Environment overview

Health
Pending

Domain
Comp4442-lab4-env.eba-57cb2z2q.us-east-1.elasticbeanstalk.com

Environment ID
e-akx27hiyig

Application name
Comp4442-lab4

Platform

Change version

Platform
Python 3.9 running on 64bit Amazon Linux 2023/4.9.1

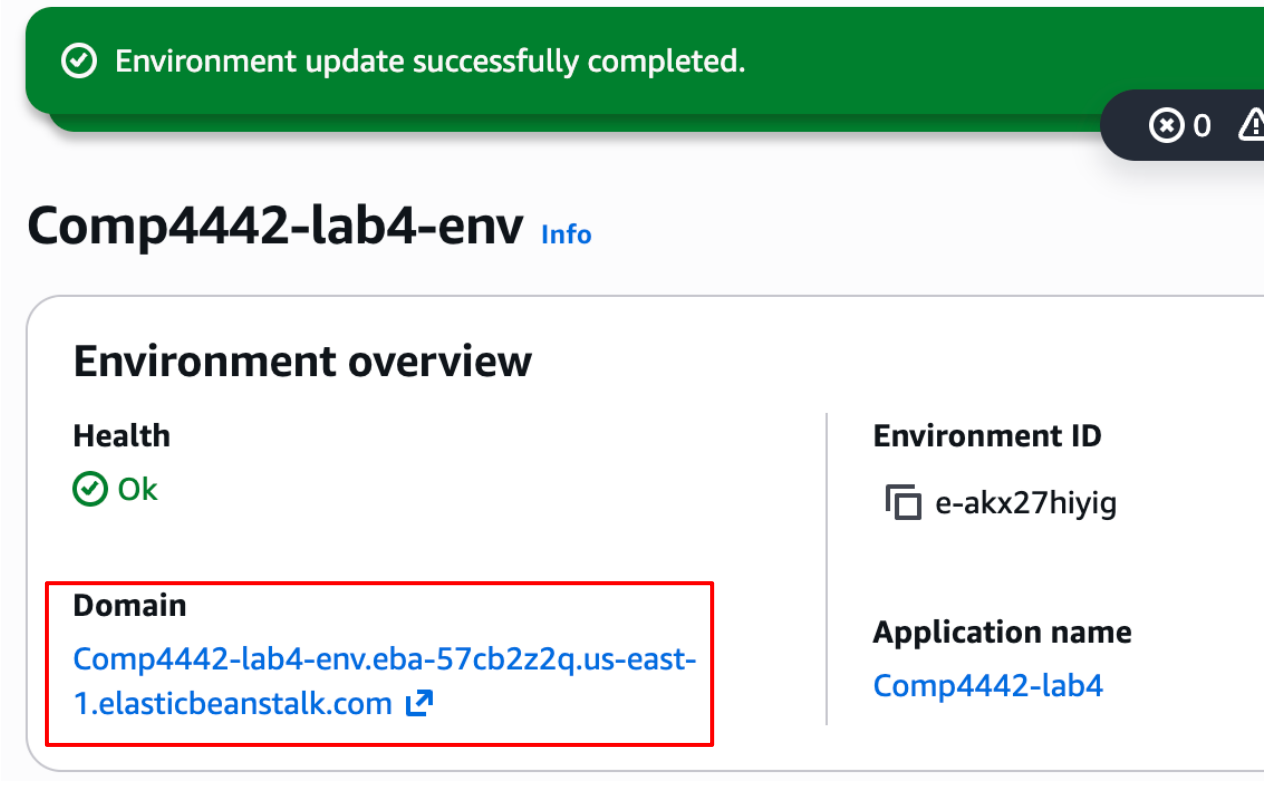
Running version
–

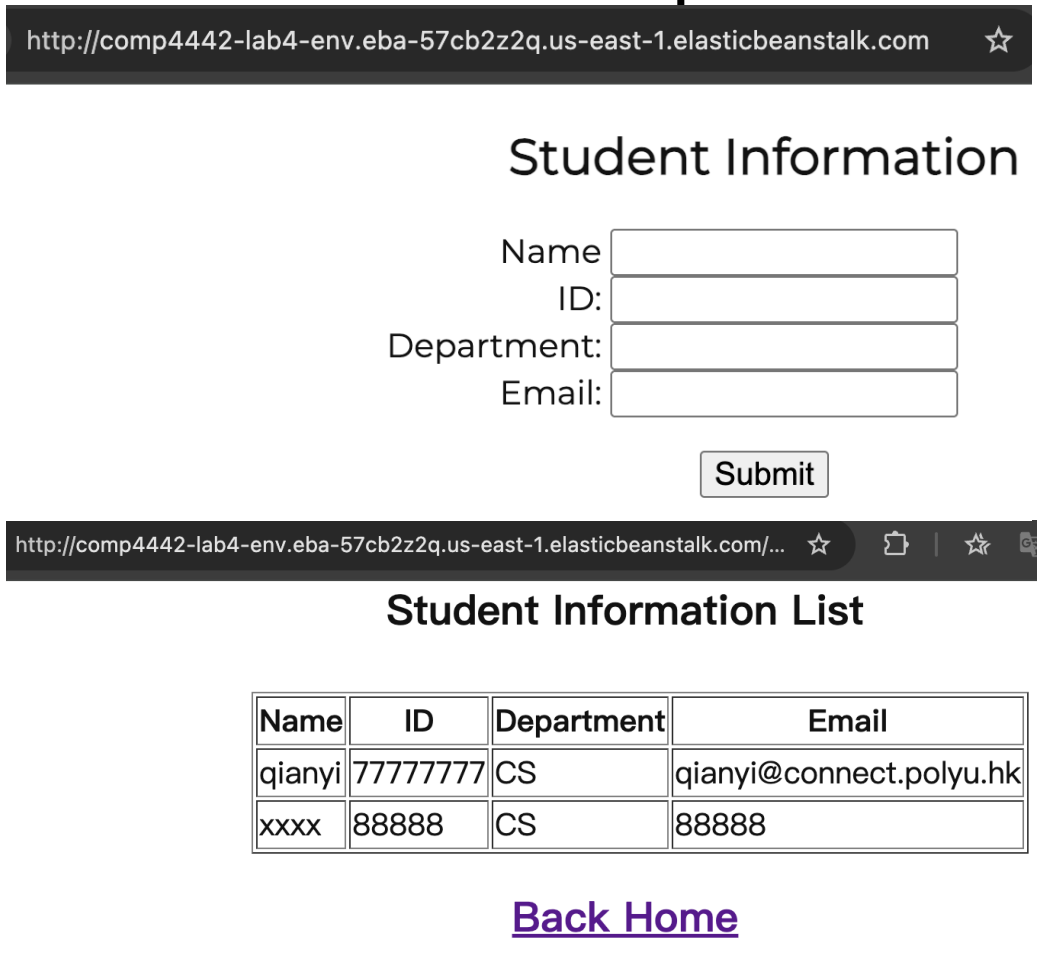
Platform state
Supported

Events | Deployments | Health | Logs | Monitoring | Alarms | Managed updates | Tags

Deploy Code in AWS Beanstalk

- Or you can create a new Beanstalk environment and upload the ZIP file as we do in lab-01.





Deploy Code in AWS Beanstalk

- Version Conflict：
  - If you use the requirements.txt generated locally, please pay attention to the version of Python in AWS Beanstalk's which should be matched to your PC's Python when you create a new Beanstalk environment.
  - The following versions have been verified.

requirements.txt

```
blinker==1.7.0
click==8.1.7
Flask==2.3.3
itsdangerous==2.1.2
Jinja2==3.1.2
MarkupSafe==2.1.3
mysql-connector-python==8.0.33
Werkzeug==2.3.7
```

Platform Change version

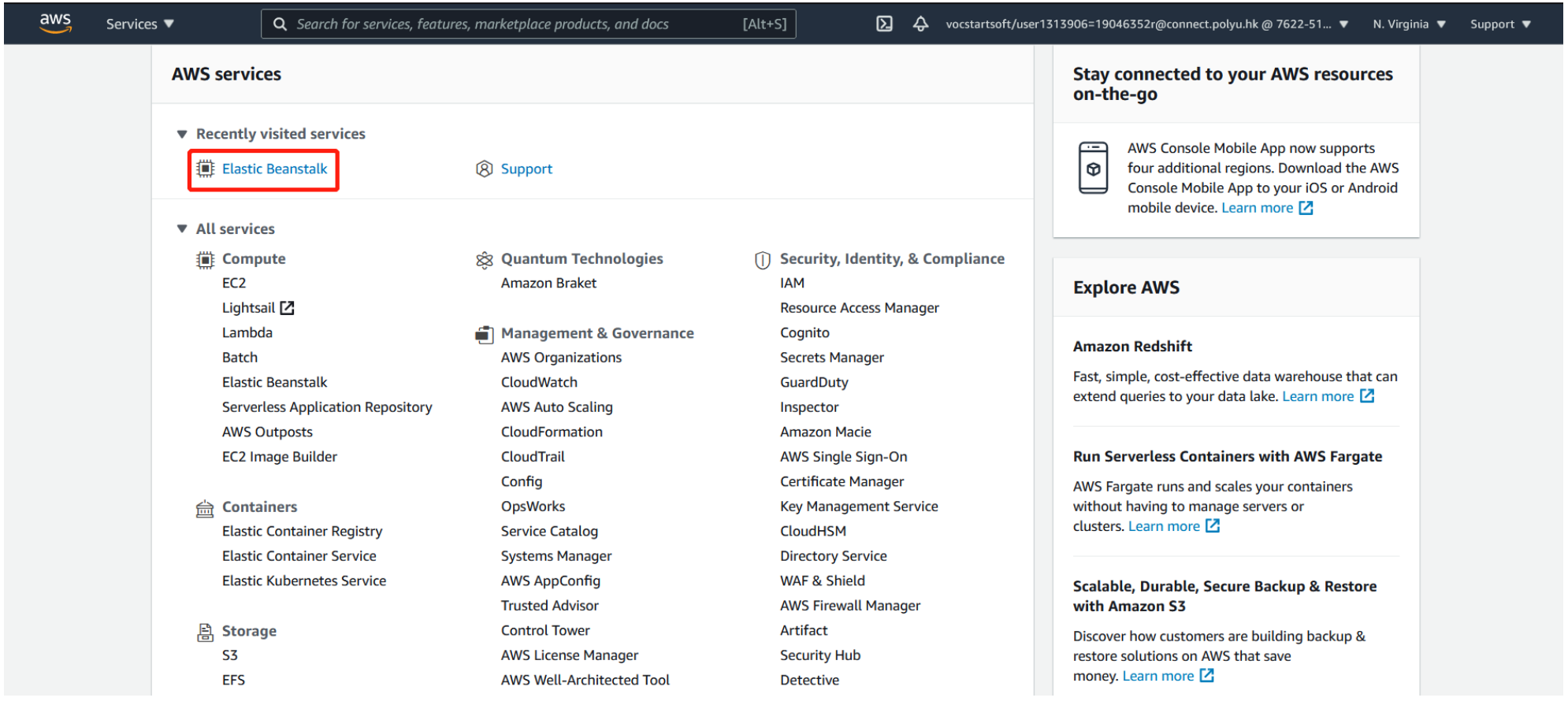
Platform

Python 3.9 running on 64bit Amazon Linux 2023/4.9.1

Q&A

Stop Elastic Beanstalk

- Access the beanstalk



Stop Elastic Beanstalk

- Terminate the environment

aws Services Search for services, features, marketplace products, and docs [Alt+S] vocstartsoft/user1313906=19046352r@connect.polyu.hk @ 7622-51... N. Virginia Support

Elastic Beanstalk

Environments
Applications
Change history

Recent environments
Comp4442lab1-env

Elastic Beanstalk > Environments

All environments

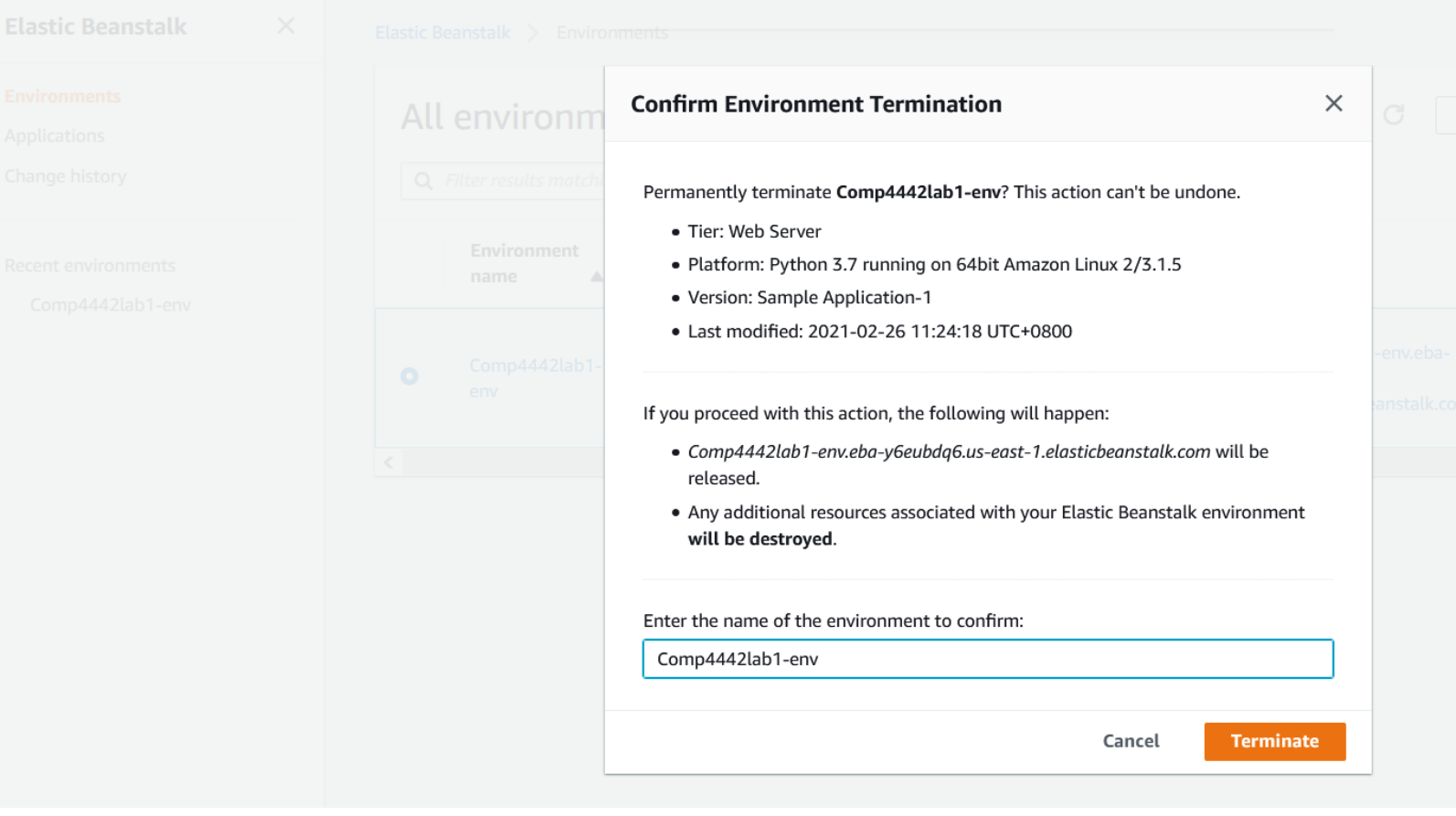
Filter results matching the display values

2 Actions Create a new environment

Load configuration
Save configuration
Swap environment URLs
Clone environment
Abort current operation
Restart app server(s)
Rebuild environment
3 Terminate environment

1

Environment name	Health	Application name	Date created	Last modified	URL	Platform	Platfo state
Comp4442lab1-env	Ok	COMP4442-lab1	2021-02-26 11:12:43 UTC+0800	2021-02-26 11:24:18 UTC+0800	Comp4442lab1-env y6eubdq6.us-east-1.elasticbeanst	Python 3.7 running on 64bit Amazon Linux 2	Supp

Stop Elastic Beanstalk

- Confirm your operation